Programación de Sistemas

Reglas de tipos para IMP++

Material de consulta para la práctica 3

Semestre 2026-2

1. Punto de partida: el ASA desazucarado

Estas reglas se aplican al árbol de sintaxis abstracta (ASA) que devuelve `parser.analiza`. Antes de iniciar la verificación, el parser ya ejecutó la transformación `desugar`. Por ello, el dominio del verificador es:

$$
\begin{aligned}
E ::= {} & \texttt{NUM}(n) \mid \texttt{BOOL}(b) \mid \texttt{LOC}(X) \mid \texttt{PLUS}(E,E) \mid \texttt{MINUS}(E,E) \mid \texttt{TIMES}(E,E) \mid \\
& \texttt{EQ}(E,E) \mid \texttt{LE}(E,E) \mid \texttt{NOT}(E) \mid \texttt{AND}(E,E) \mid \texttt{OR}(E,E) \mid \texttt{NEG}(E), \\
C ::= {} & \texttt{SKIP} \mid \texttt{ASIG}(\texttt{LOC}(X),E) \mid \texttt{SEQ}(C,C) \mid \texttt{IF}(E,C,C) \mid \texttt{WHILE}(E,C) \mid \\
& \texttt{DECL}(\tau,\texttt{LOC}(X),E) \mid \texttt{PRINT}(E) \mid \texttt{BLOQUE}([C_1,\ldots,C_n]) \mid \texttt{PROGRAMA}([C_1,\ldots,C_n]).
\end{aligned}
$$

Los nodos `INC`, `DEC` y `FOR` no pertenecen a este dominio: desaparecen durante el desazucarado. En particular,

$$
\begin{aligned}
X\texttt{++} &\rightsquigarrow \texttt{ASIG}(\texttt{LOC}(X), \texttt{PLUS}(\texttt{LOC}(X), \texttt{NUM}(1))), \\
X\texttt{--} &\rightsquigarrow \texttt{ASIG}(\texttt{LOC}(X), \texttt{MINUS}(\texttt{LOC}(X), \texttt{NUM}(1))).
\end{aligned}
$$

Un ciclo `for` se transforma en una secuencia y un `WHILE`. El verificador no define reglas adicionales para estas tres formas de azúcar.

2. Tipos, contexto y juicios

Los tipos del lenguaje son:

$$\tau ::= \texttt{int} \mid \texttt{bool}.$$

El contexto de tipos Γ es un mapa finito de identificadores a tipos. En esta práctica se usa un solo contexto. Las declaraciones explícitas se registran antes de aplicar las reglas; un identificador de IMP sin declaración explícita conserva el tipo implícito `int`. Un identificador declarado más de una vez produce un diagnóstico al construir Γ.

Como en la nota 10, se utilizan dos juicios:

$$\Gamma \vdash E : \tau \qquad \text{y} \qquad \Gamma \vdash C \ \mathsf{ok}.$$

El primero afirma que E tiene tipo τ en Γ. El segundo afirma que el comando C está bien tipificado.

3. Reglas para expresiones

3.1 Literales e identificadores

$$\frac{}{\Gamma \vdash \texttt{NUM}(n) : \texttt{int}}\ \textsc{Num} \qquad \frac{b \in \{\texttt{true}, \texttt{false}\}}{\Gamma \vdash \texttt{BOOL}(b) : \texttt{bool}}\ \textsc{BoolLit}$$

$$\frac{\Gamma(X) = \tau}{\Gamma \vdash \texttt{LOC}(X) : \tau}\ \textsc{Loc}$$

3.2 Operaciones aritméticas

$$\frac{\Gamma \vdash E_1 : \texttt{int} \qquad \Gamma \vdash E_2 : \texttt{int} \qquad K \in \{\texttt{PLUS}, \texttt{MINUS}, \texttt{TIMES}\}}{\Gamma \vdash K(E_1, E_2) : \texttt{int}}\ \textsc{Arit}$$

$$\frac{\Gamma \vdash E : \texttt{int}}{\Gamma \vdash \texttt{NEG}(E) : \texttt{int}}\ \textsc{Neg}$$

3.3 Comparaciones

$$\frac{\Gamma \vdash E_1 : \texttt{int} \qquad \Gamma \vdash E_2 : \texttt{int}}{\Gamma \vdash \texttt{LE}(E_1, E_2) : \texttt{bool}}\ \textsc{Le}$$

$$\frac{\Gamma \vdash E_1 : \tau \qquad \Gamma \vdash E_2 : \tau \qquad \tau \in \{\texttt{int}, \texttt{bool}\}}{\Gamma \vdash \texttt{EQ}(E_1, E_2) : \texttt{bool}}\ \textsc{Eq}$$

La igualdad exige que ambos operandos tengan el mismo tipo. No existe una regla que permita comparar un `int` con un `bool`.

3.4 Operaciones lógicas

$$\frac{\Gamma \vdash E : \texttt{bool}}{\Gamma \vdash \texttt{NOT}(E) : \texttt{bool}}\ \textsc{Not}$$

$$\frac{\Gamma \vdash E_1 : \texttt{bool} \qquad \Gamma \vdash E_2 : \texttt{bool} \qquad K \in \{\texttt{AND}, \texttt{OR}\}}{\Gamma \vdash K(E_1, E_2) : \texttt{bool}}\ \textsc{Log}$$

4. Reglas para comandos

4.1 Comandos básicos

$$\frac{}{\Gamma \vdash \texttt{SKIP ok}}\ \textsc{Skip}$$

$$\frac{\Gamma(X) = \tau \qquad \Gamma \vdash E : \tau}{\Gamma \vdash \texttt{ASIG}(\texttt{LOC}(X), E)\ \texttt{ok}}\ \textsc{Asig}$$

$$\frac{\Gamma \vdash E : \tau}{\Gamma \vdash \texttt{DECL}(\tau, \texttt{LOC}(X), E)\ \texttt{ok}}\ \textsc{Decl}$$

En Decl, τ es la anotación del nodo y la asociación $X : \tau$ ya fue incorporada a Γ por la pasada que registra las declaraciones. La regla comprueba el inicializador.

$$\frac{\Gamma \vdash E : \tau \qquad \tau \in \{\texttt{int}, \texttt{bool}\}}{\Gamma \vdash \texttt{PRINT}(E)\ \texttt{ok}}\ \textsc{Print}$$

4.2 Composición y estructuras de control

$$\frac{\Gamma \vdash C_1\ \texttt{ok} \qquad \Gamma \vdash C_2\ \texttt{ok}}{\Gamma \vdash \texttt{SEQ}(C_1, C_2)\ \texttt{ok}}\ \textsc{Seq}$$

$$\frac{\Gamma \vdash E : \texttt{bool} \qquad \Gamma \vdash C_1\ \texttt{ok} \qquad \Gamma \vdash C_2\ \texttt{ok}}{\Gamma \vdash \texttt{IF}(E, C_1, C_2)\ \texttt{ok}}\ \textsc{If}$$

$$\frac{\Gamma \vdash E : \texttt{bool} \qquad \Gamma \vdash C\ \texttt{ok}}{\Gamma \vdash \texttt{WHILE}(E, C)\ \texttt{ok}}\ \textsc{While}$$

Para $K \in \{\texttt{BLOQUE}, \texttt{PROGRAMA}\}$:

$$\frac{\Gamma \vdash C_i\ \texttt{ok} \quad \text{para cada } i \in \{1, \ldots, n\}}{\Gamma \vdash K([C_1, \ldots, C_n])\ \texttt{ok}}\ \textsc{Lista}$$

Cuando $n = 0$, la regla no tiene premisas: un bloque o programa vacío está bien tipificado. `BLOQUE` no cambia Γ.

5. Lectura operacional de las reglas

El verificador recorre el ASA siguiendo sus constructores:

- para una expresión, calcula el tipo de los hijos y comprueba las premisas de la regla correspondiente;
- para un comando, comprueba sus expresiones y recorre sus comandos hijos;
- si una premisa de tipos no se satisface, registra un diagnóstico;
- una expresión ya errónea se propaga sin generar diagnósticos repetidos en todos sus nodos exteriores.

La marca `Tipo.ERROR` sirve para esa propagación dentro del programa verificador. No forma parte de la gramática τ ni agrega reglas al sistema formal.